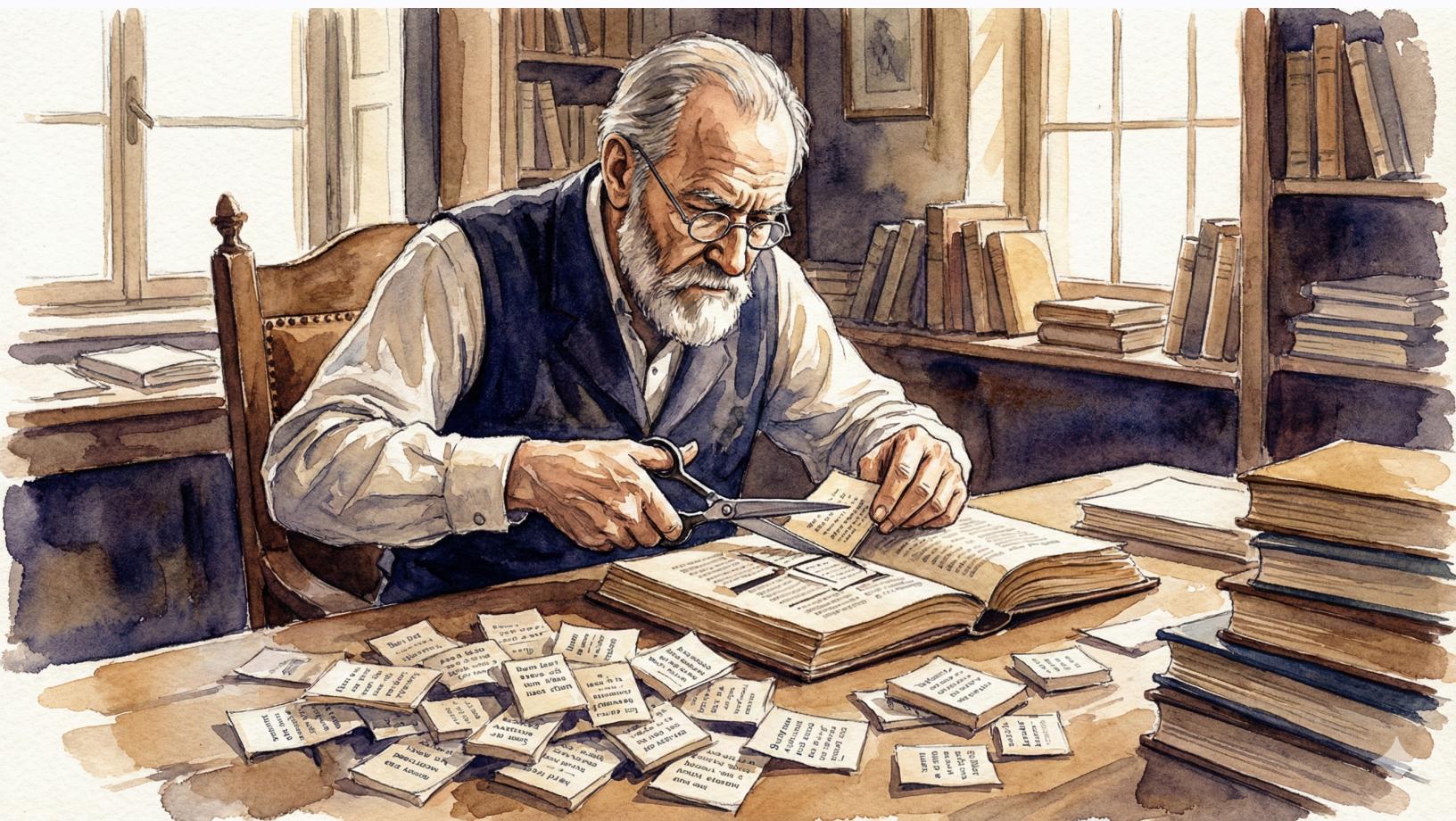


Week 4 Lesson Plan

Enterprise RAG: The Understudy — Search-Native Text and the Art of the Derivative Artifact

ASIF QAMAR



pepal leaves publisher

www.supportvectors.ai

All rights reserved. The contents in this document are the intellectual property of SupportVectors. No part of it can be shared at all without the explicit permission of the author.

First draft, June 27, 2026

The Understudy: When the Text You Search Is Not the Text You Show

The map is not the territory.

Alfred Korzybski

If I had more time, I would
have written a shorter letter.

(attributed to Blaise Pascal)

A Gentle Re-Entry

LAST WEEK WE MEASURED THE SIZE OF THOUGHTS. We chunked. We argued about where a thought begins and ends, catalogued the pathologies of cutting text into pieces, and weighed the remedies: semantic boundaries, hierarchical chunks, contextual retrieval, late chunking. We even asked the heretical question — whether to chunk at all, given that a vision-language model can embed the page as an image and skip parsing entirely.

BUT NOTICE WHAT EVERY ONE OF THOSE TECHNIQUES QUIETLY ASSUMED. Each was still indexing the *original text*. Semantic chunking cut the original differently; contextual retrieval decorated the original with a sentence of context; late chunking pooled the original in latent space. The substrate never changed. We were always searching the words the author wrote.

This week we abandon that assumption, and the whole posture of the course shifts with it. We stop asking “how do I cut the document well?”

Recommended re-read before today:
the Calibration Note of the chunking
chapter. Its last line is the hinge we
swing on this week.

and start asking a stranger, more powerful question:

What if the best thing to search is text the document never literally said?

HERE IS THE SINGLE THREAD TO HOLD. A document is written for a reader — to be argued, defended, taught, or filed away. It is almost never written to be *found* by a short, blunt query typed into a search box. So before we index a document, we will manufacture new textual objects from it — **derivative artifacts** — each one written, this time, for the searcher. We will search those. And then, having found the right place, we will show the reader the original. That last sentence is the entire week compressed: *we search derived artifacts; we show original results.*

What Must You Carry Forward

WHEN TODAY'S SESSION CONCLUDES, you should be able to do each of the following without hand-waving:

- Explain why a text's *purpose* and *audience* make it hostile to retrieval, and name three genres (papers in IMRaD form, legal prose, corporate prose) and the reader each was written for.
- State the difference between *search-native* text and raw text, and give an example where the raw text and the ideal search artifact share almost no words.
- Define a **factoid** (proposition) as the smallest self-contained claim, and extract the factoids latent in a paragraph that never states them as standalone sentences.
- Explain *embedding dilution* geometrically: why a multi-claim paragraph embeds near the centroid of its claims, far from any one of them, and why that lowers similarity to a pointed query.
- Describe a **passage rewrite** and a **QA pair**, and say precisely which retrieval failure each repairs.
- Articulate the core pattern — *retrieve the derivative, generate from the source* — and explain why it gives you findability without sacrificing fidelity or citation.
- Explain why we *add* derivative indices rather than replacing the raw-chunk index, and what each representation is paying rent for.

- Describe abstractive summarization as a change of altitude, not a compression, and name the thematic queries that only a summary can answer.
- Trace the RAPTOR algorithm — cluster, summarize, recurse — and explain what a leaf, a middle node, and the root each retrieve.
- Explain why RAPTOR's upper nodes become *definitive articles* synthesized across many documents, and why that is a property no single chunk could ever have.

If you can do these, you are ready for next week, when we climb from the single document to the whole corpus, and the clustering you saw inside RAPTOR returns wearing the formal robes of network analysis and community detection.

Act I: The Purpose Problem

ACT I MAKES ONE CLAIM AND MAKES IT STICK: text resists search not by accident but by design, because every text was written for someone, and that someone was never the search engine. We will feel this before we fix it — first through a story, then through the genres you read every day.

The Exam That Taught Me About Indexing

LET ME TELL YOU ABOUT TWO STUDENTS PREPARING FOR THE IIT ENTRANCE EXAMINATION — the gateway exam that, in India, decides futures. Call them Meera and Ravi. Meera reads physics to *understand* it. She lingers on Newton's third law until she can feel the symmetry of forces in her bones; she derives, she doubts, she rebuilds. Ravi does something different. Ravi has a coach, and the coach has a stack of ten thousand past questions. Ravi drills until, the instant he sees “two blocks, one pulley, coefficient of friction μ ,” his hand is already moving. He is not understanding the problem. He is *matching* it.

Who is right? It is a trick question — they have different purposes, and purpose dictates representation. Meera is building a model of the world. Ravi is building an *index* from question-shapes to answer-procedures. If you tested them on deep conceptual transfer, Meera wins. If you tested them on “answer two hundred questions in three hours,” Ravi wins, and it is not close.

This is not a parable against Ravi. Under a three-hour exam with two hundred questions, his representation is the correct one for his purpose. The error would be to confuse his purpose with Meera's.

NOW HEAR THE PUNCHLINE FOR OUR COURSE. A textbook is written for Meera. It builds understanding cumulatively, page after page, each idea resting on the last. Retrieval-augmented generation, at query time, is Ravi: it needs to look at a short question and instantly reach the passage that answers it. When you index a Meera-text for a Ravi-task, you get a mismatch — and that mismatch is the silent cause of half the bad RAG demos you have ever seen. The fix is not to read the textbook better. The fix is to manufacture Ravi’s flashcards from it, and index *those*.

Every Text Wears a Costume

TEXT IS NEVER NEUTRAL. It is dressed for an audience and a purpose, and the costume changes everything about how it can be found. Consider the same underlying finding — a drug reduced thirty-day mortality by four percent — as it would be written in six different rooms:

The paper hedges. In IMRaD form — Introduction, Methods, Results, and Discussion — the *number* lives in Results, its *meaning* lives in Discussion, and the Abstract softens both with “may suggest a modest benefit.” The paper is written for a skeptical peer reviewer, so it distributes its claim across structure and qualifies it into safety.

The legal brief forecloses. It is written for an adversary, so it anticipates objections and buries the operative clause in “notwithstanding the foregoing.”

The earnings call consensus-softens. It is written for shareholders and lawyers at once, so responsibility is distributed and every claim wears a helmet.

The press release sells. The textbook teaches. The tweet provokes.

Six costumes, one truth, and here is the thing that should unsettle you: a user’s query — “does the drug reduce deaths?” — matches *none* of them well. The query is naked. The text is in costume. Retrieval is the awkward party where the naked query must recognize a friend through six disguises.

Search Is an Audience Nobody Wrote For

SO WE ARRIVE AT THE WEEK’S QUIET SCANDAL. **Search-native text** — text shaped so that a blunt query lands on it — is almost never the

A game to open with. Hand each table the same one-line finding and a hidden *audience card* (Nature methods section, pharma legal, BuzzFeed listicle, earnings script, textbook, tweet). Each writes one paragraph; the room guesses the audience. The reveal lands itself: one truth wore six costumes, and not one was written to be *found*.

raw text. The raw text was written for Meera, for the reviewer, for the shareholder, for the adversary. The searcher is an audience who arrived after the writing was done, speaking a different dialect: short, declarative, impatient, phrased as a question.

The raw text and the ideal search artifact can share almost no words. “Can the licensor end the contract early?” shares nothing lexically with “notwithstanding the provisions of Section 12(b), the Licensor retains the irrevocable right to terminate.” Same meaning; disjoint vocabulary. An embedding model can bridge *some* of that gap — that is what it is for — but we are about to see, in a live bake-off, exactly how much gap is left when we lean on the embedder alone.

The Central Provocation

HERE IS THE DESIGN PRINCIPLE THAT THE REST OF THE WEEK OBEYS.

Do not wait for the user’s query to match your documents.

Manufacture documents that match your user’s queries.

If that feels like cheating, good — the best engineering often does. We are not gaming the retriever; we are *aligning* it. And there is a precise, geometric reason we must, which we state now as intuition and prove with our bodies in Act II.

A paragraph that asserts n distinct claims does not embed near any one of them. To a first approximation its embedding behaves like the average of its constituent meanings,

$$\mathbf{e}_{\text{para}} \approx \frac{1}{n} \sum_{i=1}^n \mathbf{e}_i,$$

where each $\mathbf{e}_i$ is the embedding of one self-contained claim. A pointed query $\mathbf{q}$ — “what is the population of Berlin?” — sits near one vertex $\mathbf{e}_k$. Its similarity to the paragraph is then a *diluted* quantity,

$$\cos(\mathbf{q}, \mathbf{e}_{\text{para}}) \approx \frac{1}{n} \sum_{i=1}^n \cos(\mathbf{q}, \mathbf{e}_i) \leq \cos(\mathbf{q}, \mathbf{e}_k),$$

because the one term that matters, $\cos(\mathbf{q}, \mathbf{e}_k)$, is averaged down by $n - 1$ irrelevant neighbours. Treat this as a heuristic, not a theorem — real pooling is nonlinear — but the direction is exactly right, and you will watch it happen on the floor of the classroom. Factoid extraction is the antidote: index $\mathbf{e}_k$ on its own, and the dilution disappears.

The mean of several semantically distinct vectors lands in the interior of their convex hull — away from every vertex. The paragraph embedding is that interior point. This is not a flaw in the embedding model; it is arithmetic.

Act II: The Derivative Artifacts — and the Proof

ACT II IS WHERE WE BUILD THE TOOLKIT AND THEN PUT IT ON TRIAL. We meet four derivative artifacts — factoids, rewrites, QA pairs, and the pattern that binds them — but we do not take their superiority on faith. Between meeting factoids and meeting rewrites, we run a bake-off on a book you respect, and we feel the geometry with our feet.

Factoids: Distilling the Atoms of Meaning

A FACTOID IS THE SMALLEST TEXTUAL UNIT THAT STILL MEANS SOMETHING. The chemistry analogy is more than decorative: a molecule is the smallest unit that retains the properties of a substance. Split water and you get hydrogen and oxygen — useful, but no longer wet. A factoid is the semantic molecule: split it and you have fragments that are no longer *about* anything.

Consider a paragraph from a geography text:

Berlin, situated on the banks of the River Spree, serves as the capital of Germany and is its most populous city, with a metropolitan population exceeding six million residents. The city has been a center of European politics since reunification in 1990.

This single paragraph hides at least five factoids: Berlin sits on the Spree; Berlin is the capital of Germany; Berlin is Germany’s most populous city; Berlin’s metropolitan population exceeds six million; Berlin has been a political center since 1990. Each is a complete, standalone claim — and not one of them appears in the paragraph as an isolated sentence. The phrase “the capital of Germany” is welded into a longer clause. Extraction is the act of *prying it loose* and making it self-contained, resolving the pronouns and the implicit references so that the factoid can stand alone in the index with no memory of its paragraph.

THE CRAFT LIVES IN THAT SELF-CONTAINMENT. A lazy extractor returns “It is the capital” — useless, because “it” has been severed from “Berlin.” A good extractor returns “Berlin is the capital of Germany.” This is why factoid extraction is the part of the pipeline that rewards prompt optimization most: the prompts we use in production are the product of heavy DSPy-driven tuning over our own corpora, precisely because the difference between a self-contained factoid and a dangling fragment is the difference between a retrievable thought and noise.

Key reference. Proposition-level indexing is formalized in Chen et al., “Dense X Retrieval: What Retrieval Granularity Should We Use?” Their FactoidWiki corpus yields about 2.25 propositions per sentence on average — a measure of how much meaning ordinary prose compresses, and how many retrieval targets passage-level indexing hides.

The PRML Bake-Off

NOW WE STOP ASSERTING AND START MEASURING. Our corpus today is a book you know and revere: Christopher Bishop’s *Pattern Recognition and Machine Learning* — PRML. We take its opening, the polynomial curve-fitting narrative of §1.1 through the model-selection and bias–variance material of the first three chapters, and we ask it one of its own central questions:

Why does polynomial regression overfit on a small dataset?

HERE IS THE EXPERIMENT, AND YOU WILL RUN IT YOURSELVES. First, build the naive baseline: parse PRML, chunk it, embed the chunks. Ask our question. Watch what comes back — very likely a single chunk describing the wiggling curve, or the regularization formula, but rarely the self-contained causal claim “a high-degree polynomial has more free parameters than data points, so it fits the noise, and its coefficients blow up.” That sentence exists *nowhere* in Bishop as a sentence. It is distributed across a figure, a table, and three paragraphs.

Now build the derivative indices over the same chapters. Have your group generate factoids and three-to-eight QA pairs per chunk — “Q: why do the weights become large when the model is too flexible? A: ...” — embed those, and ask again. The retrieved object is now query-shaped and self-contained. The difference, projected side by side on the screen, is meant to be *palpable* — the same visceral gap you felt last week between the col-qwen3 page-image path and the parse-chunk-embed path.

THREE RULES KEEP THE BAKE-OFF HONEST, because a strong embedder on a lexically rich query can make the naive baseline look better than it is, and a deflated demo teaches nothing:

Use queries with real vocabulary mismatch. Bishop writes “coefficients become large”; a student asks “why do the weights blow up when the curve gets too wiggly?” The derived path has normalized that dialect; the raw chunks have not. That is where the gap widens.

Include synthesis queries. “What is the relationship between model complexity, dataset size, and overfitting here?” Its answer spans three chunks. Raw chunking returns one fragment; the derived path — and, later, RAPTOR — assembles the whole.

PRML is the ideal villain, and worth naming as such to the room. Bishop never answers this in one sentence. He *shows* it: he fits $M = 0, 1, 3, 9$; displays the $M = 9$ curve thrashing through every point; tabulates the coefficients w^* exploding to enormous magnitudes; and only then introduces regularization. The causal answer is the whole arc of §1.1. It is a textbook written for Meera, and we are about to ask it a Ravi question.

Bring a pre-generated artifact set in your pocket. You run the weekend room solo, and you do not want a slow group's extraction step stalling the bake-off. The fallback set is insurance, not a crutch.

This is the embodied twin of the dilution inequality in §. The students *become* the vectors; the centroid is a place you can stand; the shrinking distance is a thing you can see. Tape on the floor is all the apparatus you need.

Score answer-ability, not topical relevance. The question is not “did a related chunk come back” but “could you actually *answer*, self-contained, from what came back?” That is the metric under which derivatives win honestly.

AND SAY THE QUIET PART OUT LOUD: *of course we still search the raw chunks.* The bake-off is not derived-*instead-of*-raw; it is derived-*in-addition-to*-raw. The raw-chunk index never leaves the system. Some queries — “what does §4.2 actually say?” — are best served by the original passage, and the raw index is exactly where they land. We are not demolishing last week’s work. We are giving it colleagues.

The Centroid Tug-of-War

WHY DID THE BAKE-OFF GO THE WAY IT DID? We could prove it on a whiteboard. Instead we prove it with our bodies, because the geometry of Act I becomes obvious the moment it is the geometry of a room. Tape out a patch of floor as embedding space. Five volunteers are *fact-vertices* — one each for “on the Spree,” “capital,” “most populous,” “six million,” “political center since 1990” — and they stand spread apart, because their meanings are distinct. Now bring in the *paragraph*: a sixth student who, by the rule of the game, must stand at the *average* of the five positions. They end up marooned in the middle of the patch, an arm’s length from everyone and close to no one.

NOW A QUERY WALKS IN. “What is the population of Berlin?” — played by a seventh student — enters and reaches for its nearest neighbour. With only the paragraph indexed, the closest thing in the room is that lonely centroid student, stranded in the middle. The match is distant; you can see the reach. Then we *factoid-extract*: the paragraph dissolves, its five vertices step forward as independent points, and the query walks *straight up* to the “six million” vertex. The distance collapses in front of the whole class. That shrinking gap is $\cos(\mathbf{q}, \mathbf{e}_k) \geq \cos(\mathbf{q}, \mathbf{e}_{\text{para}})$ made kinesthetic — and it is the mechanism on which the entire week rests.

Passage Rewrites: The Source’s Understudy

NOT ALL TEXT IS WRITTEN TO BE FOUND. Academic prose is written to be argued, legal prose to be defended, corporate prose to be forgotten.

A **passage rewrite** reformulates a span in plain, direct, search-friendly language — without changing its meaning. We keep the original; the rewrite is a *shadow document*, indexed alongside it, invisible to the user but highly visible to the retriever.

Take the clause:

Notwithstanding the provisions of Section 12(b), the Licensor retains the irrevocable right to terminate this Agreement upon thirty (30) calendar days' written notice.

A search-friendly rewrite:

The licensor can terminate the agreement with 30 days' written notice, regardless of Section 12(b).

The rewrite embeds closer to how a user would ask — “can the licensor end the agreement early?” — yet the original clause, with all its operative force, remains the authoritative passage handed to the generation model. This is the understudy who auditions for the retriever so that the star can take the stage to perform.

QA Pairs: Planting Query-Shaped Objects

OF ALL THE ARTIFACTS, THE QUESTION-ANSWER PAIR IS THE MOST MISCHIEVOUS — AND THE MOST DIRECT ATTACK ON THE COSTUME PROBLEM. The idea is simple: given a chunk, have an LLM generate the plausible questions that chunk could answer, paired with their answers, and embed *both*.

Why does it work? Because retrieval is a matching problem, and matching is easiest when both sides speak the same dialect. A dense retriever compares a *query* embedding with *document* embeddings. Ordinarily this is a cross-genre comparison — a question measured against declarative prose. But if the indexed object was itself *derived from a question*, the comparison becomes same-genre: question against question. You have planted a query-shaped decoy that the retriever *should* find.

A good QA pipeline generates three to eight questions per chunk, varied in specificity (from “what does this section discuss?” to “what learning rate did experiment 3 use?”), in formulation (keyword-ish to natural-language), and in abstraction (factual to interpretive). Each pair stores a pointer back to its source chunk — because at generation time the model must see the chunk, not the synthetic question. And note the failure mode you will inspect in lab: a careless synthetic question can *mislead* the retriever, promising an answer the source does not actually contain. Generation hygiene matters.

The rewrite carries a real risk: simplification can shed precision. “Irrevocable” is a legal term of art the plain rewrite drops. The risk is defused by the core pattern of §: the rewrite only *finds* the clause; the *original* is what the generator reads and cites. The rewrite is the bait; the original is the evidence.

HyDE, the mirror image. QA pairs make the *index* question-shaped at ingestion time. HyDE makes the *query* document-shaped at query time: it asks an LLM to hallucinate a hypothetical answer, embeds *that*, and retrieves against it. Both close the same genre gap from opposite ends. Worth raising when a student asks “why not just fix the query instead?” — the answer is: you can do both.

The Core Pattern: Retrieve the Derivative, Generate from the Source

ONE PATTERN UNIFIES EVERYTHING IN THIS ACT, and you should learn it once and use it everywhere:

Retrieve against the derivative. Generate from the source.
Every derivative artifact carries a pointer home.

This pattern is also what makes derivative artifacts *safe*. Because the source is always what the model reads and cites, a clumsy rewrite or an over-eager factoid can cost you a retrieval miss but never a fabricated citation. The blast radius of a bad derivative is bounded.

The factoid, the rewrite, the QA pair — each is a better lure than the raw chunk for some class of query, and each is a worse thing to actually quote. So we never quote them. They win the retrieval; the original passage gives the testimony. This is also the clean resolution of the Meera-and-Ravi tension from §: we never threw the textbook away. Ravi’s flashcards *index* the understanding; Meera’s text still *is* the understanding. The exam-cram sheet did not replace learning. It pointed back to it.

Act III: Altitude — Summaries, RAPTOR, and the Corpus

EVERYTHING IN ACT II WAS LOCAL. Factoids, rewrites, and QA pairs operate at the altitude of a single passage. But some questions cannot be answered from any one passage, however well we extract it — because their answer is not *in* the document; it is *about* the document. For those we must gain altitude, and then, at the very end, we must leave the single document behind entirely.

Abstractive Summarization: Trading Detail for Altitude

A SUMMARY IS NOT A COMPRESSION. IT IS A CHANGE OF ALTITUDE. The word “compression” suggests the same content packed tighter into a smaller box. Summarization is not that. It is ascent in a balloon: you rise, and large structures invisible from the ground swim into view — the shape of the coastline, the pattern of the fields — while the fine details that filled your vision on the ground vanish. You trade resolution for scope. Neither view is truer. They are different altitudes over the same land.

This is why summaries are the *complement* of factoids, never their replacement. Factoids sacrifice context for precision; summaries sacrifice precision for context. Some queries are unanswerable from the ground:

Philip Anderson’s “More Is Different” names the principle exactly: at each level of scale, qualitatively new properties appear that are invisible at the level below. A document’s theme is an emergent property of its chunks — present in none of them, visible only from altitude. Hold this thought; in two subsections it becomes the whole point.

“what is this report about?”, “what are the main findings?”, “how does this policy differ from the last version?” No single chunk contains the answer, because the answer is distributed across the whole. Only a summary — an artifact manufactured to live at altitude — can be retrieved in response.

The engineering decisions are three: *granularity* (summarize per chunk, per section, per document, per corpus — each a different artifact with different retrieval behaviour); *faithfulness* (an abstractive summary can hallucinate, and for RAG a confident false summary is worse than none — faithfulness must be enforced by prompting, verification, or citation requirements); and *indexing* (summaries usually live in their own namespace, because their retrieval characteristics are too different from chunks to blend).

Summaries at Many Granularities

ONCE YOU ACCEPT THAT ALTITUDE IS A DIAL, THE NATURAL QUESTION IS: WHY CHOOSE ONE SETTING? A per-section summary answers “what does the methods section cover?”; a per-document summary answers “what is this paper’s contribution?”; a per-corpus summary answers “what is this whole collection about?” Each is cheap to generate relative to its value, and each catches a band of thematic queries the others miss. The instinct of the week — one source, many representations — applies along the altitude axis just as it did along the precision axis. And once you have summaries at several fixed altitudes, you are one small step from making altitude *continuous*. That step is RAPTOR.

RAPTOR: The Zoom Lens

RAPTOR TAKES SUMMARIZATION AND GIVES IT RECURSIVE STRUCTURE. The algorithm is elegant enough to state in five lines:

1. Start with leaf-level chunks.
2. Cluster nearby chunks by embedding similarity.
3. Summarize each cluster into a new node.
4. Repeat: cluster the summaries, summarize the clusters.
5. Stop at a single root (or a few high-level nodes).

The result is a tree. The leaves are fine-grained chunks; the middle nodes are progressively more abstract summaries; the root is a synopsis

Key reference. Sarthi et al., “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval.” The clustering uses soft, overlapping Gaussian-mixture assignment over UMAP-reduced embeddings — a chunk may belong to more than one cluster, because a paragraph can be about more than one thing. File that detail away: soft clustering of a corpus is a doorway we walk through next week.

The Matryoshka parallel is exact. A Matryoshka embedding is one vector meaningful at many dimensionalities; RAPTOR is one corpus retrievable at many resolutions. Matryoshka gives multi-resolution *vectors*; RAPTOR gives multi-resolution *text*.

This is Anderson's "More Is Different" cashed out in retrieval. The corpus-level article is an emergent object: it exists in none of the source documents and could not, because it is a property of the collection, not of any member. We are, quite literally, manufacturing the encyclopedia entry the corpus never wrote about itself.

of the whole. At retrieval time you search *every* level at once. A factual query — "what learning rate?" — matches a leaf. A thematic query — "what is the overall methodology?" — matches a middle node. A global query — "what is this collection about?" — matches the root. The query chooses the magnification; the index is a zoom lens, not a fixed focal length.

Picture a galaxy. From inside, you see individual stars — the leaves. Pull back and the stars blur into spiral arms — the middle nodes. Pull back further and the whole galaxy is a single bright coin — the root. No magnification is the true one; each answers a different question about the same object.

From One Book to a Library: RAPTOR's Definitive Articles

HERE IS THE IDEA I MOST WANT YOU TO CARRY OUT OF THE ROOM TODAY. Run RAPTOR over a *single* document and its upper nodes are summaries of that document. But run it over a *corpus* — a hundred papers, a thousand support tickets, every internal memo on a topic — and the clusters no longer respect document boundaries. A cluster gathers the chunks that are *about the same thing*, drawn from wherever they live. Its summary is therefore something new: not a digest of any one source, but a *synthesis across all of them* — a definitive article on a topic that no single document contains.

Imagine asking a shelf — PRML and Hastie–Tibshirani–Friedman's *Elements of Statistical Learning* together — "what is overfitting?" A naive system returns Bishop's best chunk or ESL's best chunk. A corpus-level RAPTOR node returns one coherent answer that has already reconciled both treatments into a single, canonical paragraph. That is the definitive-article property, and it is the reason this technique is not merely "summarization with extra steps."

AND THIS IS EXACTLY WHERE WE STOP — ON PURPOSE. To build that corpus-level article well, you eventually want to cluster not by raw embedding proximity but by *relational structure*: which entities and ideas are genuinely connected across documents. That is community detection over a graph, and it is a different and beautiful machinery — network analysis, modularity, the Leiden algorithm, and the move "from local to global" — that deserves its own day. Notice, though, that you have already met its seed twice today: once in RAPTOR's soft clustering of a corpus, and once in the simple observation that a corpus has a structure no single document can see. Next week the seed

becomes a forest. We will meet GraphRAG, and its memory-augmented cousin, and the network science that makes them work.

For now, hold the whole week in one breath:

Text is written for a reader. Search needs an artifact written for the query. So we manufacture those artifacts, we search them — alongside the raw chunks, never instead of them — and we show the reader the original.

Laboratory Sessions: What We Build and Measure

THIS WEEK'S LABS ARE GENERATION-HEAVY AT INGESTION AND EVALUATION-HEAVY AT QUERY TIME. By the end, every participant should have built at least two derivative-artifact pipelines and *measured* their effect on retrieval — because the claim of the week is empirical, and you should never take an empirical claim on faith when the measurement costs an afternoon.

Lab 1: The PRML Bake-Off, Instrumented

THE BAKE-OFF FROM § BECOMES A MEASURED EXPERIMENT. Ingest the first three chapters of PRML three ways: (a) a raw-chunk index; (b) a factoid index; (c) a QA-pair index. Keep all three live — remember, we search the raw chunks too — and fuse their results.

Assemble a query set of twenty questions in three bands: pointed factual (“what degree polynomial does Bishop fit in Figure 1.4?”), vocabulary-mismatch (“why do the weights blow up when the curve gets too wiggly?”), and synthesis (“how do model complexity and dataset size jointly drive overfitting?”). For each arm, record Recall@k, MRR, and — the metric that matters most — a human *answer-ability* judgement: could you write a correct, self-contained answer from what came back? Tabulate. The expected shape: raw chunks hold their own on the pointed band, derivatives pull ahead on the mismatch band, and the synthesis band is where everyone struggles and RAPTOR (Lab 3) earns its keep.

Hold the embedding model, the chunker, and the top-*k* fixed across all three arms. The only variable that may change is the *representation*. A bake-off that quietly changes two things at once teaches nothing.

Lab 2: Factoid and QA Extraction Pipelines

BUILD THE TWO LOCAL EXTRACTORS AND INSPECT THEIR OUTPUT BY EYE. Write an LLM prompt that decomposes a chunk into self-contained

factoids, and a second that generates three-to-eight QA pairs per chunk. Run both over a fifty-chunk sample. Then do the thing engineers skip and should not: *read the artifacts*. Find the factoid that still says “it” instead of “Berlin.” Find the synthetic question whose answer the source does not actually contain. The craft of self-containment (§) and the failure mode of misleading QA pairs (§) are invisible in aggregate metrics and obvious on the page.

Lab 3: A RAPTOR Tree at Two Altitudes

IMPLEMENT THE CLUSTER-SUMMARIZE-RECURSE LOOP FOR TWO OR THREE LEVELS. Over the same PRML chapters, build leaves from chunks, cluster, summarize the clusters, and cluster once more. Then aim the synthesis queries from Lab 1 at different levels of the tree and watch where they land — the pointed queries at the leaves, the thematic ones at the middle nodes. Measure whether tree depth actually improves recall for the synthesis band. Keep the cost ledger open: how many summarization calls did two levels cost, and what would a nightly rebuild cost at corpus scale?

Lab 4: The Multi-Representation Tournament

FINALLY, ASSEMBLE EVERY INDEX AND ASK WHICH ONES ARE PAYING RENT. Combine raw chunks, factoids, QA pairs, and RAPTOR nodes behind one fused retriever. Then *ablate*: remove each representation in turn and measure the system-level drop. The deliverable is an honest table — one row per representation, one column per query band — showing exactly where each artifact earns its storage and its ingestion cost. Some representation will surprise you by contributing almost nothing on your corpus; that is a finding, not a failure, and it is the kind of finding that separates an engineer from a cargo-culter.

The Day's Rhythm

IF YOU WANT THE SHAPE OF THE SESSION IN A SINGLE GLANCE, here it is. We open with a *frame* (Meera and Ravi; the six costumes), so that the problem is felt before any technique appears. We then meet the *toolkit* (factoids), interrupt it immediately with the *proof* (the PRML bake-off and the Centroid Tug-of-War), and only then complete the toolkit (rewrites, QA pairs, the core pattern) now that no one doubts

it is needed. We climb to *altitude* (summaries, RAPTOR), and we end by stepping off the single document onto the *corpus*, where RAPTOR's upper nodes become definitive articles — and where next week begins. Surprise first, mechanism second, principle last: the order is the pedagogy.

Frame Why raw text resists search: purpose, audience, IMRaD, the naked query.

Toolkit, part one Factoids as the atoms; embedding dilution as the enemy.

Proof The PRML bake-off (palpable difference) and the Centroid Tug-of-War (why).

Toolkit, part two Rewrites, QA pairs, and “retrieve the derivative, generate from the source.”

Altitude Abstractive summaries; RAPTOR's zoom lens.

Corpus Definitive articles synthesized across documents — the bridge to GraphRAG.

Essential Readings

THESE ARE REQUIRED. Each develops a thread we opened today.

- **Chen et al., *Dense X Retrieval: What Retrieval Granularity Should We Use?***

The definitive study on proposition-level retrieval and the source of the factoid idea. Establishes that retrieval granularity is not a fixed given but a design variable with measurable consequences. Read it to understand why an atom beats a paragraph for pointed queries.

- **Sarathi et al., *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval*.**

Introduces the hierarchical summarization tree. The key insight is that retrieval resolution should match query abstraction. Pay attention to the soft clustering — it is the hinge to next week.

- **Bishop, *Pattern Recognition and Machine Learning*, §1.1–3.2.**

Our bake-off corpus. Re-read the polynomial curve-fitting narrative specifically as a *retrieval* problem: notice how the answer to “why does this overfit?” is distributed across a figure, a coefficient table,

and several paragraphs, and never stated as one self-contained claim. That is the whole motivation for derivative artifacts, written by a master who was not thinking about search at all.

- **Ma et al., *Multi-Vector Retrieval as a Multi-Representation Learning Problem*.**

Formalizes the principle behind the entire week: a single document should be represented by multiple vectors (or multiple texts), each capturing a different aspect. Read it after the bake-off, when the principle is no longer abstract.

Oliver Twist's List of Readings

OPTIONAL, BUT EACH ONE DEEPENS THE WEEK. For the voracious mind.

- **Gao et al., *Precise Zero-Shot Dense Retrieval without Relevance Labels* (HyDE, 2022).**

The mirror image of QA-pair extraction: instead of making the index question-shaped, make the query document-shaped. Read it to see the genre-gap problem solved from the other end.

- **Wang et al., *Generating Synthetic Training Data for Dense Retrieval* (InPars).**

The QA-generation strategy has roots in synthetic data generation for retrieval. Understand the lineage, and the failure modes of synthetic questions.

- **Dhuliawala et al., *Chain-of-Verification Reduces Hallucination in Large Language Models*.**

Faithfulness in summarization is non-negotiable for RAG. One concrete approach to verifying LLM-generated summaries before you trust them in an index.

- **Asai et al., *Self-RAG: Learning to Retrieve, Generate, and Critique*.**

Explores a self-correcting retrieve-generate loop — directly relevant to quality control of the derivative artifacts we manufacture.

- **Anderson, *More Is Different* (1972).**

Four pages of physics that explain why a corpus has properties no document has. The intellectual license for the definitive-article idea, and a good thing to have read before next week's leap to the whole graph.

A Calibration Note

THIS SESSION WAS DENSE, AND IT TURNED ON A SINGLE INVERSION. For three weeks we treated the document as the thing to be indexed. Today we treated it as raw material — a substrate from which to manufacture better things to index. We met four local artifacts (factoids, rewrites, QA pairs, and the pattern that keeps them safe), and two altitudes (summaries, and RAPTOR’s continuous zoom), and we ended by leaving the single document behind for the corpus-level article that no document contains.

The stories were not decoration. Meera and Ravi were the whole argument: representation follows purpose, and the searcher’s purpose is not the reader’s. The six costumes were the same argument in a different room. And the Centroid Tug-of-War was not a game bolted onto the mathematics — it *was* the mathematics, standing up and walking around. The equation $\cos(\mathbf{q}, \mathbf{e}_k) \geq \cos(\mathbf{q}, \mathbf{e}_{\text{para}})$ and the student walking from the marooned centroid to the “six million” vertex are the same sentence, told twice.

Text is written for a reader; search needs an artifact written for the query. So we manufacture the artifact, we search it — alongside the raw chunks, never in their place — and we show the reader the original. We retrieve the derivative. We generate from the source.

Next week we climb from the book to the library. The soft clustering hiding inside RAPTOR, and the corpus-level article we glimpsed at the end, return as network analysis and community detection. We will meet GraphRAG — and its memory-augmented cousin, Memory-GraphRAG — and discover that “find the communities, then summarize each one” is the same instinct as today’s, scaled from one document’s paragraphs to a whole corpus’s ideas. Bring the prism. We are about to hand it a much larger light.

References

- Anderson, Philip W. (1972). “More Is Different”. In: *Science* 177.4047, pages 393–396. DOI: [10.1126/science.177.4047.393](https://doi.org/10.1126/science.177.4047.393). URL: <https://doi.org/10.1126/science.177.4047.393>.
- Asai, Akari et al. (2023). **Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection**. DOI: [10.48550/arXiv.2310.11511](https://arxiv.org/abs/2310.11511). arXiv: [2310.11511](https://arxiv.org/abs/2310.11511) [cs.CL]. URL: <https://arxiv.org/abs/2310.11511>.
- Bishop, Christopher M. (2006). **Pattern Recognition and Machine Learning**. Information Science and Statistics. The polynomial curve-fitting narrative of §1.1 is our bake-off corpus: a text written to build understanding, not to answer a point query. New York: Springer.
- Chen, Tong et al. (2024). **Dense X Retrieval: What Retrieval Granularity Should We Use?** Proposes proposition-level retrieval granularity. DOI: [10.48550/arXiv.2312.06648](https://arxiv.org/abs/2312.06648). arXiv: [2312.06648](https://arxiv.org/abs/2312.06648) [cs.CL]. URL: <https://arxiv.org/abs/2312.06648>.
- Dhuliawala, Shehzaad et al. (2023). **Chain-of-Verification Reduces Hallucination in Large Language Models**. DOI: [10.48550/arXiv.2309.11495](https://arxiv.org/abs/2309.11495). arXiv: [2309.11495](https://arxiv.org/abs/2309.11495) [cs.CL]. URL: <https://arxiv.org/abs/2309.11495>.
- Gao, Luyu et al. (2023). “Precise Zero-Shot Dense Retrieval without Relevance Labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 1762–1777.
- Kusupati, Ajay et al. (2022). “Matryoshka Representation Learning”. In: **Advances in Neural Information Processing Systems**. NeurIPS 2022. DOI: [10.48550/arXiv.2205.13147](https://arxiv.org/abs/2205.13147). URL: <https://arxiv.org/abs/2205.13147>.
- Sarathi, Parth et al. (2024). “RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval”. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. Hierarchical summarization tree for multi-level abstractive retrieval. URL: <https://arxiv.org/abs/2401.18059>.
- Wang, Liang, Nan Yang, Xiaolong Huang, et al. (2023). **SimLM: Pre-training with Representation Bottleneck for Dense Passage Retrieval**. Synthetic query generation for retrieval training. DOI: [10.48550/arXiv.2207.02578](https://arxiv.org/abs/2207.02578). arXiv: [2207.02578](https://arxiv.org/abs/2207.02578) [cs.IR]. URL: <https://arxiv.org/abs/2207.02578>.
- Wang, Liang, Nan Yang, and Furu Wei (2023). **Query2doc: Query Expansion with Large Language Models**. Multi-representation retrieval via query expansion. DOI: [10.48550/arXiv.2303.07678](https://arxiv.org/abs/2303.07678). arXiv: [2303.07678](https://arxiv.org/abs/2303.07678) [cs.IR]. URL: <https://arxiv.org/abs/2303.07678>.

There were 19 pages in this document.

FEEDBACK

Please send any feedback on this document, or report errors to the author at asif@supportvectors.com

